

QUESTION 1:

HackerRank

[Prepare](#) [Java](#) [Introduction](#) [Welcome to Java!](#)

Exit Full Screen View

Problem

Welcome to the world of Java! In this challenge, we practice printing to stdout.

The code stubs in your editor declare a Solution class and a main method. Complete the main method by copying the two lines of code below and pasting them inside the body of your main method.

```
System.out.println("Hello, World.");
System.out.println("Hello, Java.");
```

Input Format

There is no input for this challenge.

Output Format

You must print two lines of output:

1. Print Hello, World, on the first line.

2. Print Hello, Java, on the second line.

Sample Output

```
Hello, World.
Hello, Java.
```

Change Theme Language Java 7

```
1 public class Solution {
2     public static void main(String[] args) {
3         System.out.println("Hello, World.");
4         System.out.println("Hello, Java.");
5     }
6 }
7
```

Line: 7 Col: 1

QUESTION 2:

HackerRank

[Prepare](#) [Java](#) [Introduction](#) [Java Stdin and Stdout I](#)

Exit Full Screen View

Problem

Most HackerRank challenges require you to read input from `stdin` (standard input) and write output to `stdout` (standard output).

One popular way to read input from `stdin` is by using the `Scanner` class and specifying the Input Stream as `System.in`. For example:

```
Scanner scanner = new Scanner(System.in);
String myString = scanner.next();
int myInt = scanner.nextInt();
scanner.close();

System.out.println("myString is: " + myString);
System.out.println("myInt is: " + myInt);
```

The code above creates a Scanner object named `scanner` and uses it to read a String and an int. It then closes the Scanner object because there is no more input to read, and prints to stdout using `System.out.println(String)`. So, if our input is:

```
Hi 5
```

Our code will print:

```
myString is: Hi
myInt is: 5
```

Alternatively, you can use the `BufferedReader` class.

Change Theme Language Java 7

```
1 import java.util.*;
2
3 public class Solution {
4     public static void main(String[] args) {
5         Scanner scan = new Scanner(System.in);
6         int a = scan.nextInt();
7         int b = scan.nextInt();
8         int c = scan.nextInt();
9         System.out.println(a);
10        System.out.println(b);
11        System.out.println(c);
12        scan.close();
13    }
14 }
15
16
```

Line: 16 Col: 1

QUESTION 3:

Problem

Submissions

Leaderboard

Discussions

Task

Given an integer, n , perform the following conditional actions:

- If n is odd, print **Weird**
- If n is even and in the inclusive range of 2 to 5, print **Not Weird**
- If n is even and in the inclusive range of 6 to 20, print **Weird**
- If n is even and greater than 20, print **Not Weird**

Complete the stub code provided in your editor to print whether or not n is weird.

Input Format

A single line containing a positive integer, n .

Constraints

- $1 \leq n \leq 100$

Output Format

Print **Weird** if the number is weird; otherwise, print **Not Weird**.

Sample Input 0

3

Change Theme

Language

Java 7

Exit Full Screen View

```
1  import java.io.*;
2  import java.math.*;
3  import java.security.*;
4  import java.text.*;
5  import java.util.*;
6  import java.util.concurrent.*;
7  import java.util.regex.*;
8  public class Solution {
9
10     private static final Scanner scanner = new Scanner(System.in);
11     public static void main(String[] args) {
12         int N = scanner.nextInt();
13         if (N%2!=0){
14             System.out.println("Weird");
15         }
16         else{
17             if (N>=2 && N<=5){
18                 System.out.print("Not Weird");
19             }
20             else if (N>=6 && N<=20){
21                 System.out.print("Weird");
22             }
23             else{
24                 System.out.print("Not Weird");
25             }
26         }
27     }
28 }
```

Line: 8 Col: 24

Upload Code as File

Test against custom input

Run Code

Submit Code

QUESTION 4:

Problem

Submissions

Leaderboard

Discussions

Task

In this challenge, you must read an integer, a double, and a String from stdin, then print the values according to the instructions in the Output Format section below. To make the problem a little easier, a portion of the code is provided for you in the editor.

Note: We recommend completing [Java Stdin and Stdout I](#) before attempting this challenge.

Input Format

There are three lines of input:

1. The first line contains an integer.
2. The second line contains a double.
3. The third line contains a String.

Output Format

There are three lines of output:

1. On the first line, print **String:** followed by the unaltered String read from stdin.
2. On the second line, print **Double:** followed by the unaltered double read from stdin.
3. On the third line, print **Int:** followed by the unaltered integer read from stdin.

To make the problem easier, a portion of the code is already provided in the editor.

Note: If you use the `nextLine()` method immediately following the `nextInt()` method, recall that `nextInt()` reads integer tokens; because of this, the last newline character for that line of integer input is still queued in the input buffer and the next `nextLine()` will be reading the remainder of the integer line (which is empty).

Sample Input

Change Theme

Language

Java 7

Exit Full Screen View

```
1  import java.util.Scanner;
2
3  public class Solution {
4
5      public static void main(String[] args) {
6          Scanner scan = new Scanner(System.in);
7          int i = scan.nextInt();
8          double d=scan.nextDouble();
9          scan.nextLine();
10         String s=scan.nextLine();
11
12         System.out.println("String: " + s);
13         System.out.println("Double: " + d);
14         System.out.println("Int: " + i);
15         scan.close();
16     }
17 }
18
```

Line: 18 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

QUESTION 5:

HackerRank

[Prepare](#) [Java](#) [Introduction](#) [Java Output Formatting](#)

[Exit Full Screen View](#)

Problem

Java's `System.out.printf` function can be used to print formatted output. The purpose of this exercise is to test your understanding of formatting output using `printf`.

To get you started, a portion of the solution is provided for you in the editor; you must format and print the input to complete the solution.

Input Format

Every line of input will contain a String followed by an integer.

Each String will have a maximum of 10 alphabetic characters, and each integer will be in the inclusive range from 0 to 999.

Output Format

In each line of output there should be two columns:

The first column contains the String and is left justified using exactly 15 characters.

The second column contains the integer, expressed in exactly 3 digits; if the original input has less than three digits, you must pad your output's leading digits with zeroes.

Sample Input

java 100
cpp 65
python 50

Sample Output

=====

Change Theme Language Java 7

```
1 import java.util.Scanner;  
2  
3 public class Solution {  
4  
5     public static void main(String[] args) {  
6         Scanner sc=new Scanner(System.in);  
7         System.out.println("=====");  
8         for(int i=0;i<3;i++){  
9             String s1=sc.next();  
10            int x=sc.nextInt();  
11            System.out.printf("%-15s%03d\n",s1,x);  
12        }  
13        System.out.println("=====");  
14    }  
15 }  
16  
17  
18  
19  
20
```

Line: 20 Col: 1

☐ Test against custom input

QUESTION 6:

HackerRank

[Prepare](#) [Java](#) [Introduction](#) [Java Loops I](#)

[Exit Full Screen View](#)

Problem

Objective

In this challenge, we're going to use loops to help us do some simple math.

Task

Given an integer, N , print its first 10 multiples. Each multiple $N \times i$ (where $1 \leq i \leq 10$) should be printed on a new line in the form: $N \times i = \text{result}$.

Input Format

A single integer, N .

Constraints

- $2 \leq N \leq 20$

Output Format

Print 10 lines of output; each line i (where $1 \leq i \leq 10$) contains the **result** of $N \times i$ in the form:

$N \times i = \text{result}$.

Sample Input

2

Sample Output

$2 \times 1 = 2$
 $2 \times 2 = 4$

Change Theme Language Java 7

```
1 import java.io.*;  
2 import java.math.*;  
3 import java.security.*;  
4 import java.text.*;  
5 import java.util.*;  
6 import java.util.concurrent.*;  
7 import java.util.regex.*;  
8  
9  
10  
11 public class Solution {  
12     public static void main(String[] args) throws IOException {  
13         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(  
14             System.in));  
15  
16         int N = Integer.parseInt(bufferedReader.readLine().trim());  
17         for(int i=1;i<=10;i++){  
18             System.out.println(N+" x "+i+" = "+N*i);  
19         }  
20         bufferedReader.close();  
21     }  
22 }
```

Line: 22 Col: 1

☐ Test against custom input

QUESTION 7:

Problem

Submissions

Leaderboard

Discussions

We use the integers a , b , and n to create the following series:

$$(a + 2^0 \cdot b), (a + 2^0 \cdot b + 2^1 \cdot b), \dots, (a + 2^0 \cdot b + 2^1 \cdot b + \dots + 2^{n-1} \cdot b)$$

You are given q queries in the form of a , b , and n . For each query, print the series corresponding to the given a , b , and n values as a single line of n space-separated integers.

Input Format

The first line contains an integer, q , denoting the number of queries.

Each line i of the q subsequent lines contains three space-separated integers describing the respective a_i , b_i , and n_i values for that query.

Constraints

- $0 \leq q \leq 500$
- $0 \leq a, b \leq 50$
- $1 \leq n \leq 15$

Output Format

For each query, print the corresponding series on a new line. Each series must be printed in order as a single line of n space-separated integers.

Sample Input

```
2
0 2 10
5 3 5
```

Change Theme Language Java 7

```
1 import java.util.*;
2 import java.io.*;
3
4 class Solution{
5     public static void main(String []argh){
6         Scanner in = new Scanner(System.in);
7         int t=in.nextInt();
8         for(int i=0;i<t;i++){
9             int a = in.nextInt();
10            int b = in.nextInt();
11            int n = in.nextInt();
12            int sum=a;
13            for(int j=0;j<n;j++){
14                sum+=(1<<j)*b;
15                System.out.print(sum+" ");
16            }
17            System.out.println();
18        }
19        in.close();
20    }
21 }
22
```

Line: 22 Col: 1

Upload Code as File Test against custom input

Run Code Submit Code

QUESTION 8:

Problem

Submissions

Leaderboard

Discussions

Java has 8 primitive data types: char, boolean, byte, short, int, long, float, and double. For this exercise, we'll work with the primitives used to hold integer values (byte, short, int, and long):

- A byte is an 8-bit signed integer.
- A short is a 16-bit signed integer.
- An int is a 32-bit signed integer.
- A long is a 64-bit signed integer.

Given an input integer, you must determine which primitive data types are capable of properly storing that input.

To get you started, a portion of the solution is provided for you in the editor.

Reference: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>

Input Format

The first line contains an integer, T , denoting the number of test cases.

Each test case, T , is comprised of a single line with an integer, n , which can be arbitrarily large or small.

Output Format

For each input variable n and appropriate primitive **dataType**, you must determine if the given primitives are capable of storing it. If yes, then print:

```
n can be fitted in:
* dataType
```

Change Theme Language Java 7

```
1 import java.util.*;
2 import java.io.*;
3
4 class Solution{
5     public static void main(String []argh)
6     {
7         Scanner sc = new Scanner(System.in);
8         int t=sc.nextInt();
9
10        for(int i=0;i<t;i++)
11        {
12            try
13            {
14                long x=sc.nextLong();
15                System.out.println(x+" can be fitted in:");
16                if(x>=-128 && x<=127)System.out.println(" * byte");
17                if(x>=-32768 && x<=32767)
18                    System.out.println(" * short");
19
20                if(x>=-2147483648L && x<=2147483647L)
21                    System.out.println(" * int");
22
23                System.out.println(" * long");
24            }
25            catch(Exception e)
26            {}
27        }
28    }
29 }
```

Line: 6 Col: 6

Upload Code as File Test against custom input

Run Code Submit Code

QUESTION 9:

Problem

Submissions

Leaderboard

Discussions

"In computing, End Of File (commonly abbreviated EOF) is a condition in a computer operating system where no more data can be read from a data source." — (Wikipedia: End-of-file)

The challenge here is to read n lines of input until you reach EOF, then number and print all n lines of content.

Hint: Java's `Scanner.hasNext()` method is helpful for this problem.

Input Format

Read some unknown n lines of input from `stdin(System.in)` until you reach EOF; each line of input contains a non-empty String.

Output Format

For each line, print the line number, followed by a single space, and then the line content received as input.

Sample Input

```
Hello world
I am a file
Read me until end-of-file.
```

Sample Output

```
1 Hello world
```

Change Theme Language Java 7

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6 public class Solution {
7
8     public static void main(String[] args) {
9         Scanner sc = new Scanner(System.in);
10
11         int count = 1;
12
13         while (sc.hasNextLine()) {
14             String s = sc.nextLine();
15             System.out.println(count + " " + s);
16             count++;
17         }
18
19         sc.close();
20     }
21 }
22
```

Line: 5 Col: 26

Upload Code as File

Test against custom input

Run Code

Submit Code

QUESTION 10:

Problem

Submissions

Leaderboard

Discussions

Static initialization blocks are executed when the class is loaded, and you can initialize static variables in those blocks.

It's time to test your knowledge of Static initialization blocks. You can read about it [here](#).

You are given a class `Solution` with a main method. Complete the given code so that it outputs the area of a parallelogram with breadth B and height H . You should read the variables from the standard input.

If $B \leq 0$ or $H \leq 0$, the output should be "java.lang.Exception: Breadth and height must be positive" without quotes.

Input Format

There are two lines of input. The first line contains B : the breadth of the parallelogram. The next line contains H : the height of the parallelogram.

Constraints

- $-100 \leq B \leq 100$
- $-100 \leq H \leq 100$

Output Format

If both values are greater than zero, then the main method must output the area of the parallelogram. Otherwise, print "java.lang.Exception: Breadth and height must be positive" without quotes.

Sample input 1

```
10
20
```

Change Theme Language Java 7

```
1 > import java.io.*;
2
3 static int B;
4 static int H;
5 static boolean flag = true;
6 static {
7     Scanner sc = new Scanner(System.in);
8     B = sc.nextInt();
9     H = sc.nextInt();
10    if (B <= 0 || H <= 0) {
11        flag = false;
12        System.out.println("java.lang.Exception: Breadth and height must be positive");
13    }
14 }
15
16 public static void main(String[] args){
17     if(flag){
18         int area=B*H;
19         System.out.print(area);
20     }
21
22     //end of main
23 }
24
25 //end of class
26
```

Line: 20 Col: 32

Upload Code as File

Test against custom input

Run Code

Submit Code

HackerRank

Prepare

Java

Introduction

Java Int to String

Exit Full Screen View

Problem

You are given an integer n , you have to convert it into a string.

Please complete the partially completed code in the editor. If your code successfully converts n into a string s the code will print "Good job". Otherwise it will print "Wrong answer".

n can range between -100 to 100 inclusive.

Sample Input 0

```
100
```

Sample Output 0

```
Good job
```

Submissions

Leaderboard

Discussions

Change Theme

Language

Java 7

⌵

⌵

⌵

```

4 import java.util.*; ...
13 String s=String.valueOf(n);
14
15 if (n == Integer.parseInt(s)) {
16     System.out.println("Good job");
17 } else {
18     System.out.println("Wrong answer.");
19 }
20 } catch (DoNotTerminate.ExitTrappedException e) {
21     System.out.println("Unsuccessful Termination!!");
22 }
23 }
24 }
25
26 //The following class will prevent you from terminating the code using exit(0)!
27 class DoNotTerminate {
28
29     public static class ExitTrappedException extends SecurityException {
30
31         private static final long serialVersionUID = 1;
32     }
33
34     public static void forbidExit() {
35         final SecurityManager securityManager = new SecurityManager() {
36             @Override

```

Line: 13 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

HackerRank

Prepare

Java

Introduction

Java Currency Formatter

Exit Full Screen View

Problem

Given a double-precision number, *payment*, denoting an amount of money, use the `NumberFormat` class' `getCurrencyInstance` method to convert *payment* into the US, Indian, Chinese, and French currency formats. Then print the formatted values as follows:

```
US: formattedPayment
India: formattedPayment
China: formattedPayment
France: formattedPayment
```

where *formattedPayment* is *payment* formatted according to the appropriate `Locale`'s currency.

Note: India does not have a built-in `Locale`, so you must **construct one** where the language is `en` (i.e., English).

Input Format

A single double-precision number denoting *payment*.

Constraints

- $0 \leq \text{payment} \leq 10^9$

Output Format

On the first line, print US: *u* where *u* is *payment* formatted for US currency.

On the second line, print India: *i* where *i* is *payment* formatted for Indian currency.

On the third line, print China: *c* where *c* is *payment* formatted for Chinese currency.

On the fourth line, print France: *f*, where *f* is *payment* formatted for French currency.

Submissions

Leaderboard

Discussions

Change Theme

Language

Java 7

⌵

⌵

⌵

```

1 import java.util.*;
2 import java.text.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8         double payment = scanner.nextDouble();
9         scanner.close();
10        Locale indiaLocale = new Locale("en", "IN");
11
12        NumberFormat usFormat = NumberFormat.getCurrencyInstance(Locale.US);
13        NumberFormat indiaFormat = NumberFormat.getCurrencyInstance(indiaLocale);
14        NumberFormat chinaFormat = NumberFormat.getCurrencyInstance(Locale.CHINA);
15        NumberFormat franceFormat = NumberFormat.getCurrencyInstance(Locale.FRANCE);
16
17        String us = usFormat.format(payment);
18        String india = indiaFormat.format(payment);
19        String china = chinaFormat.format(payment);
20        String france = franceFormat.format(payment);
21        System.out.println("US: " + us);
22        System.out.println("India: " + india);
23        System.out.println("China: " + china);
24        System.out.println("France: " + france);
25    }

```

Line: 27 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

HackerRank

Prepare

Java

Introduction

Java Date and Time

Exit Full Screen View

Problem

The `Calendar` class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields such as YEAR, MONTH, DAY_OF_MONTH, HOUR, and so on, and for manipulating the calendar fields, such as getting the date of the next week.

You are given a date. You just need to write the method, `getDay`, which returns the day on that date. To simplify your task, we have provided a portion of the code in the editor.

Example

```
month = 8
day = 14
year = 2017
```

The method should return *MONDAY* as the day on that date.



Submissions

Leaderboard

Discussions

Change Theme

Language

Java 7

⌵

⌵

⌵

```

9 class Result {
10
11     *
12     * The function is expected to return a STRING.
13     * The function accepts following parameters:
14     * 1. INTEGER month
15     * 2. INTEGER day
16     * 3. INTEGER year
17     */
18
19     public static String findDay(int month, int day, int year) {
20         Calendar cal = Calendar.getInstance();
21         cal.set(year, month - 1, day);
22
23         String[] days = {
24             "SUNDAY",
25             "MONDAY",
26             "TUESDAY",
27             "WEDNESDAY",
28             "THURSDAY",
29             "FRIDAY",
30             "SATURDAY"
31         };
32
33         return days[cal.get(Calendar.DAY_OF_WEEK) - 1];
34     }
35 }

```

Line: 8 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

Given a string, *s*, and two indices, *start* and *end*, print a substring consisting of all characters in the inclusive range from *start* to *end* - 1. You'll find the String class' [substring method](#) helpful in completing this challenge.

Input Format

The first line contains a single string denoting *s*.

The second line contains two space-separated integers denoting the respective values of *start* and *end*.

Constraints

- $1 \leq |s| \leq 100$
- $0 \leq start < end \leq n$
- String *s* consists of English alphabetic letters (i.e., [a - zA - Z]) only.

Output Format

Print the substring in the inclusive range from *start* to *end* - 1.

Sample Input

```
HelloWorld
3 7
```

Sample Output

```
lowo
```

Change Theme

Language

Java 7

⌵

⌵

⌵

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 public class Solution {
8
9     public static void main(String[] args) {
10         Scanner in = new Scanner(System.in);
11         String S = in.next();
12         int start = in.nextInt();
13         int end = in.nextInt();
14         System.out.println(S.substring(start,end));
15         in.close();
16     }
17 }
18
```

Line: 18 Col: 1

⬇ Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

"A string is traditionally a sequence of characters, either as a literal constant or as some kind of variable." — [Wikipedia: String \(computer science\)](#)

This exercise is to test your understanding of Java Strings. A sample String declaration:

```
String myString = "Hello World!"
```

The elements of a String are called characters. The number of characters in a String is called the length, and it can be retrieved with the `String.length()` method.

Given two strings of lowercase English letters, *A* and *B*, perform the following operations:

- Sum the lengths of *A* and *B*.
- Determine if *A* is lexicographically larger than *B* (i.e.: does *B* come before *A* in the dictionary?).
- Capitalize the first letter in *A* and *B* and print them on a single line, separated by a space.

Input Format

The first line contains a string *A*. The second line contains another string *B*. The strings are comprised of only lowercase English letters.

Output Format

There are three lines of output:

For the first line, sum the lengths of *A* and *B*.

For the second line, write Yes if *A* is lexicographically greater than *B* otherwise print No

Change Theme

Language

Java 7

⌵

⌵

⌵

```
1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7         Scanner sc=new Scanner(System.in);
8         String A=sc.next();
9         String B=sc.next();
10        System.out.println(A.length() + B.length());
11        if (A.compareTo(B) > 0) {
12            System.out.println("Yes");
13        } else {
14            System.out.println("No");
15        }
16        String A1 = A.substring(0, 1).toUpperCase() + A.substring(1);
17        String B1 = B.substring(0, 1).toUpperCase() + B.substring(1);
18        System.out.println(A1 + " " + B1);
19        sc.close();
20    }
21 }
22
23
24
25
```

Line: 26 Col: 1

⬇ Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

We define the following terms:

- Lexicographical Order**, also known as alphabetic or dictionary order, orders characters as follows:
$$A < B < \dots < Y < Z < a < b < \dots < y < z$$

For example, ball < cat, dog < dorm, Happy < happy, Zoo < ball.

- A **substring** of a string is a contiguous block of characters in the string. For example, the substrings of abc are a, b, c, ab, bc, and abc.

Given a string, *s*, and an integer, *k*, complete the function so that it finds the lexicographically smallest and largest substrings of length *k*.

Function Description

Complete the `getSmallestAndLargest` function in the editor below.

`getSmallestAndLargest` has the following parameters:

- string *s*: a string
- int *k*: the length of the substrings to find

Returns

- string: the string ' + "\n" + ' where and are the two substrings

Input Format

The first line contains a string denoting *s*.

The second line contains an integer denoting *k*.

Constraints

Change Theme

Language

Java 7

⌵

⌵

⌵

```
1 > import java.util.Scanner; ...
4
5
6 public static String getSmallestAndLargest(String s, int k) {
7     String smallest = s.substring(0, k);
8     String largest = s.substring(0, k);
9
10    for (int i = 1; i <= s.length() - k; i++) {
11        String sub = s.substring(i, i + k);
12
13        if (sub.compareTo(smallest) < 0) {
14            smallest = sub;
15        }
16
17        if (sub.compareTo(largest) > 0) {
18            largest = sub;
19        }
20    }
21
22    return smallest + "\n" + largest;
23 }
24 > ...
```

Line: 4 Col: 1

⬇ Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

We define the following terms:

- **Lexicographical Order**, also known as alphabetic or dictionary order, orders characters as follows:
$$A < B < \dots < Y < Z < a < b < \dots < y < z$$

For example, ball < cat, dog < dorm, Happy < happy, Zoo < ball.
- A **substring** of a string is a contiguous block of characters in the string. For example, the substrings of abc are a, b, c, ab, bc, and abc.

Given a string, *s*, and an integer, *k*, complete the function so that it finds the lexicographically smallest and largest substrings of length *k*.

Function Description

Complete the getSmallestAndLargest function in the editor below.

getSmallestAndLargest has the following parameters:

- string *s*: a string
- int *k*: the length of the substrings to find

Returns

- string: the string ' + "\n" + ' where and are the two substrings

Input Format

The first line contains a string denoting *s*.

The second line contains an integer denoting *k*.

Change Theme

Language

Java 7

⌕

⋮

```
1 > import java.util.Scanner; ...
4
5
6 public static String getSmallestAndLargest(String s, int k) {
7     String smallest = s.substring(0, k);
8     String largest = s.substring(0, k);
9
10    for (int i = 1; i <= s.length() - k; i++) {
11        String sub = s.substring(i, i + k);
12
13        if (sub.compareTo(smallest) < 0) {
14            smallest = sub;
15        }
16
17        if (sub.compareTo(largest) > 0) {
18            largest = sub;
19        }
20    }
21
22    return smallest + "\n" + largest;
23 }
24 > ...
```

Line: 4 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

A palindrome is a word, phrase, number, or other sequence of characters which reads the same backward or forward.

Given a string *A*, print Yes if it is a palindrome, print No otherwise.

Constraints

- *A* will consist at most 50 lower case english letters.

Sample Input

madam

Sample Output

Yes

Change Theme

Language

Java 7

⌕

⋮

```
1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7
8         Scanner sc=new Scanner(System.in);
9         String A=sc.next();
10        String rev = "";
11        for (int i = A.length() - 1; i >= 0; i--) {
12            rev = rev + A.charAt(i);
13        }
14        if (A.equals(rev)) {
15            System.out.println("Yes");
16        } else {
17            System.out.println("No");
18        }
19        sc.close();
20    }
21 }
22
23
24
25
```

Line: 26 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

Problem

Submissions

Leaderboard

Discussions

HackerRank

Prepare

Java

Strings

Java String Tokens

Exit Full Screen View

Given a string, s , matching the regular expression $[A-Za-z !,?,_'\@]+\text{}$, split the string into tokens. We define a token to be one or more consecutive English alphabetic letters. Then, print the number of tokens, followed by each token on a new line.

Note: You may find the `String.split` method helpful in completing this challenge.

Input Format

A single string, s .

Constraints

- $1 \leq \text{length of } s \leq 4 \cdot 10^5$
- s is composed of any of the following: English alphabetic letters, blank spaces, exclamation points (!), commas (,), question marks (?), periods (.), underscores (_), apostrophes ('), and at symbols (@).

Output Format

On the first line, print an integer, n , denoting the number of tokens in string s (they do not need to be unique). Next, print each of the n tokens on a new line in the same order as they appear in input string s .

Sample Input

```
He is a very very good boy, isn't he?
```

Sample Output

Change Theme

Language

Java 7

⌵

⌵

⌵

```
1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7         Scanner scan = new Scanner(System.in);
8         String s = scan.nextLine();
9         s = s.trim();
10
11         if (s.length() == 0) {
12             System.out.println(0);
13         } else {
14             String[] tokens = s.split("[ !,?._'\@]+\text{");
15
16             System.out.println(tokens.length);
17
18             for (String token : tokens) {
19                 System.out.println(token);
20             }
21
22             scan.close();
23         }
24     }
25 }
```

Line: 26 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

Problem

Submissions

Leaderboard

Discussions

HackerRank

Prepare

Java

BigNumber

Java Primality Test

Exit Full Screen View

A prime number is a natural number greater than 1 whose only positive divisors are 1 and itself. For example, the first six prime numbers are 2, 3, 5, 7, 11, and 13.

Given a large integer, n , use the Java BigInteger class' `isProbablePrime` method to determine and print whether it's prime or not prime.

Input Format

A single line containing an integer, n (the number to be checked).

Constraints

- n contains at most 100 digits.

Output Format

If n is a prime number, print `prime`; otherwise, print `not prime`.

Sample Input

```
13
```

Sample Output

```
prime
```

Explanation

The only positive divisors of 13 are 1 and 13, so we print `prime`.

Change Theme

Language

Java 7

⌵

⌵

⌵

```
1 import java.io.*;
2 import java.math.*;
3 import java.security.*;
4 import java.text.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.regex.*;
8
9
10
11
12 public class Solution {
13     public static void main(String[] args) throws IOException {
14         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
15
16         String n = bufferedReader.readLine();
17
18         BigInteger num = new BigInteger(n);
19
20         if (num.isProbablePrime(1)) {
21             System.out.println("prime");
22         } else {
23             System.out.println("not prime");
24         }
25         bufferedReader.close();
26     }
27 }
```

Line: 8 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Problem

Submissions

Leaderboard

Discussions

Problem

Submissions

Leaderboard

Discussions

HackerRank

Prepare

Java

Strings

Java Anagrams

Exit Full Screen View

Two strings, a and b , are called anagrams if they contain all the same characters in the same frequencies. For this challenge, the test is not case-sensitive. For example, the anagrams of CAT are CAT, ACT, tac, TCA, aTC, and CtA.

Function Description

Complete the `isAnagram` function in the editor.

`isAnagram` has the following parameters:

- string a : the first string
- string b : the second string

Returns

- boolean: If a and b are case-insensitive anagrams, return true. Otherwise, return false.

Input Format

The first line contains a string a .

The second line contains a string b .

Constraints

- $1 \leq \text{length}(a), \text{length}(b) \leq 50$
- Strings a and b consist of English alphabetic characters.
- The comparison should NOT be case sensitive.

Sample Input 0

```
anagram
margana
```

Change Theme

Language

Java 7

⌵

⌵

⌵

```
1 import java.util.Scanner;
2
3
4
5 static boolean isAnagram(String a, String b) {
6     a = a.toLowerCase();
7     b = b.toLowerCase();
8
9     if (a.length() != b.length()) {
10         return false;
11     }
12
13     char[] arr1 = a.toCharArray();
14     char[] arr2 = b.toCharArray();
15
16     java.util.Arrays.sort(arr1);
17     java.util.Arrays.sort(arr2);
18
19     return java.util.Arrays.equals(arr1, arr2);
20 }
21
22 public static void main(String[] args) { ... }
```

Line: 4 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

